Marketing Big Data Project

Technical documentation

Star schema on HDFS, processed with Spark, queried with Hive

28 October 2026

Contents

1. Overview
2. Architecture
3. Data model
4. Step 1: Data generation and upload
5. Step 2: Quality checks
6. Step 3: Processing with Spark
7. Step 4: Hive tables
8. Step 5: Analytical queries
9. Findings
10. Problems and how they were solved
11. How to run the project
12. Limitations and next steps

1 Overview

This project builds a complete big data workflow for marketing analytics on a single-node Hadoop environment. It starts from raw marketing events (impressions, clicks, views, add to cart and purchases) and ends with business answers such as which channel returns the most per unit of ad spend.

The data follows a star schema: one large fact table of 10 million events and four small dimension tables. The data is generated locally because no real marketing dataset of this size was available, so it is synthetic (see *Limitations*).

The work is organised in layers. Each layer reads the output of the previous one and writes its own, so every step can be checked and re-run on its own:

Step	What it does	Tool
1. Generate and upload	Creates the star schema as Parquet files and copies them to HDFS (staging)	PySpark, hdfs dfs
2. Quality checks	Checks sizes, nulls, duplicate keys, orphan keys and business rules	Spark
3. Processing	Cleans the fact table and builds three summary tables	Spark
4. Hive tables	Exposes the processed data and the dimensions as external tables	Hive (Beeline)
5. Analytical queries	Answers 10 business questions and saves each result on HDFS	HiveQL

Environment

Component	Version / setting
Platform	Single-node virtual machine (VMware Workstation), CentOS Stream 10
Hadoop	HDFS with replication factor 1, YARN
Spark	4.1.3 (Scala 2.13), run with spark-submit, local[2], 2 GB driver memory
Hive	4.2.1, Beeline in embedded mode, Derby metastore
Java	OpenJDK 21
File format	Parquet (fact table partitioned by month), results as CSV

2 Architecture

Data moves from left to right through five zones. Each zone has a fixed location so it is always clear where a file came from.

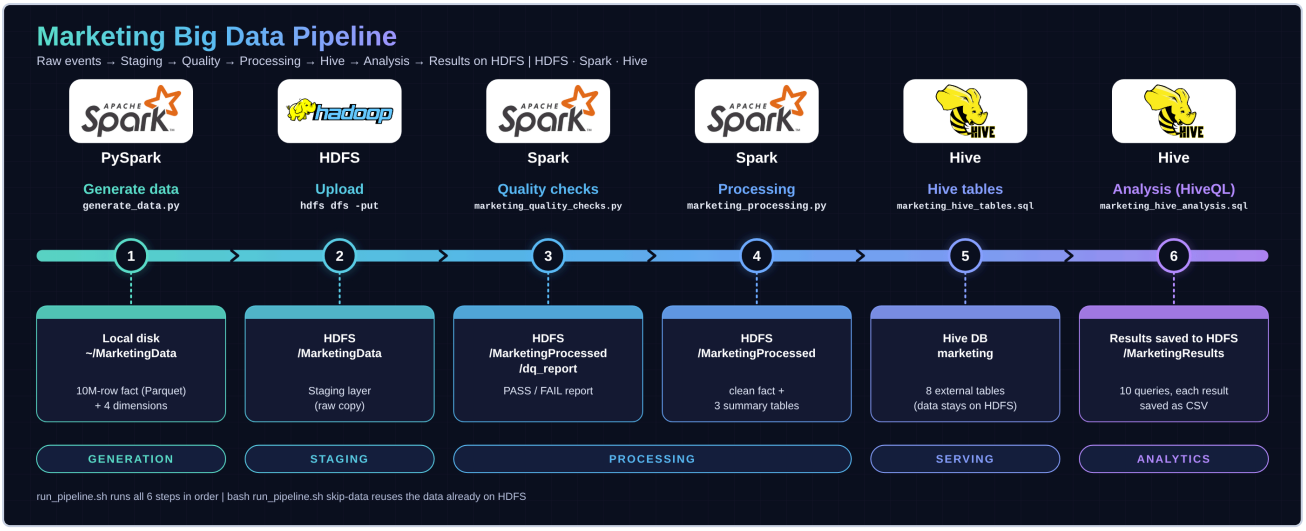


Figure 1. End-to-end pipeline: tools used in each step and where each step stores its output.

Layer	Location	Content
Local disk	~/MarketingData	Raw Parquet files as first generated
Staging	hdfs:///MarketingData	Unchanged copy of the generated data, the starting point for everything else
Processed	hdfs:///MarketingProcessed	Cleaned fact table, three summary tables and the data quality report
Serving	Hive database `marketing`	Eight external tables over the files above (data stays on HDFS)
Results	hdfs:///MarketingResults	One folder per analytical query, each holding a CSV file

Keeping a staging layer means the original data is never modified. If a processing step is wrong, it can be fixed and run again from staging without regenerating anything.

3 Data model

The model is a star schema. The fact table records one row per marketing event and points to the dimensions through keys.

Table	Rows	Key columns	Description
fact_events	10,000,000	event_id; customer_id, product_id, campaign_id	One row per event. Columns: event_date, event_type, device, price, quantity, revenue, ad_cost, year_month. Stored as Parquet, partitioned by year_month (24 partitions, 2024-01 to 2025-12).
dim_customers	500,000	customer_id	country, gender, segment (New, Regular, VIP), signup_date
dim_products	10,000	product_id	category, brand, list_price
dim_campaigns	500	campaign_id	channel (google_ads, facebook, instagram, tiktok, youtube, email), objective, budget
dim_date	730	date_key	full_date, year, quarter, month, month_name, day_of_week, is_weekend

Event types are *impression*, *click*, *view*, *add_to_cart* and *purchase*. Revenue is only recorded on purchases and ad cost is only recorded on paid events such as clicks. The date dimension was kept so that the analysis can answer calendar questions such as monthly revenue and weekend against weekday.

4 Step 1: Data generation and upload

Script: `generate_data.py`

The data is generated on the local disk first with PySpark and written as Parquet into `~/MarketingData`, one folder per table. Generating locally and then uploading keeps the generation separate from the cluster and lets the upload step be repeated on its own.

The folders are then copied to HDFS, which becomes the staging layer:

```
hdfs dfs -mkdir -p /MarketingData
hdfs dfs -put ~/MarketingData/* /MarketingData/
hdfs dfs -du -h /MarketingData
```

Data volume. The first plan was 100 million events. On the single virtual machine this overloaded the CPU and the machine crashed, so the fact table was reduced to 10 million events and Spark was limited to two local cores. The pipeline itself is unchanged, only the volume is smaller.

5 Step 2: Quality checks

Script: `marketing_quality_checks.py` **Input:** staging **Output:** `hdfs:///MarketingProcessed/dq_report`

Before any transformation, the staging data is checked. The script prints each check as a section and finally collects every result into one report with a PASS or FAIL status.

Check	What it verifies
Dataset size	Row and column count of every table
Schema and sample	Column names and types, first rows of the fact table
Null values	Number of nulls in every column of every table
Duplicate keys	event_id, customer_id, product_id, campaign_id and date_key are unique
Statistics	count, mean, standard deviation, min and max of the numeric columns
Referential integrity	Every customer, product and campaign in the fact table exists in its dimension
Business rules	Price is positive, ad cost is not negative, revenue only on purchases, every purchase has revenue, quantity only on purchases, dates inside the expected range



Figure 2. The quality step starts when run through the pipeline.

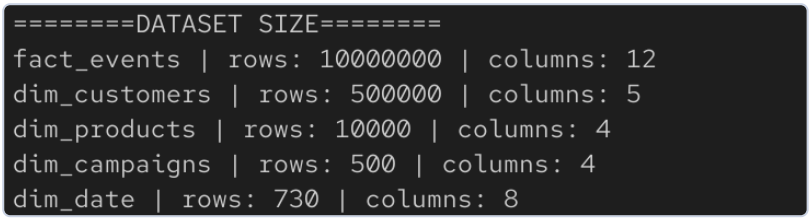


Figure 3. Dataset size: 10,000,000 events and the four dimensions.

```
========SCHEMA========
-- fact_events
root
 |-- event_id: long (nullable = true)
 |-- event_date: date (nullable = true)
 |-- customer_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- campaign_id: integer (nullable = true)
 |-- event_type: string (nullable = true)
 |-- device: string (nullable = true)
 |-- price: double (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- revenue: double (nullable = true)
 |-- ad_cost: double (nullable = true)
 |-- year_month: string (nullable = true)
```

Figure 4. Schema of the fact table.

```
========SAMPLE DATA (fact_events, first 5 rows)========
+--------+----------+-----------+----------+-----------+-----------+-------+------+--------+-------+-------+----------+
|event_id|event_date|customer_id|product_id|campaign_id| event_type| device| price|quantity|revenue|ad_cost|year_month|
+--------+----------+-----------+----------+-----------+-----------+-------+------+--------+-------+-------+----------+
| 6048597|2025-10-04|     254001|      6126|         94| impression| mobile|366.86|       3|    0.0|    0.0|   2025-10|
| 5000033|2025-10-01|      98341|      2625|        295|      click| mobile| 309.3|       1|    0.0|   0.04|   2025-10|
| 6048608|2025-10-15|     202602|      8068|          2|      click| tablet|466.89|       2|    0.0|   0.86|   2025-10|
| 5000038|2025-10-17|      96605|      2552|        110|add_to_cart|desktop|236.26|       1|    0.0|    0.0|   2025-10|
| 6048616|2025-10-20|     139085|      3547|        365|       view| tablet| 23.53|       2|    0.0|    0.0|   2025-10|
+--------+----------+-----------+----------+-----------+-----------+-------+------+--------+-------+-------+----------+
only showing top 5 rows
```

Figure 5. First five rows of the fact table.

```
========NULL VALUES PER COLUMN========
-- fact_events
-RECORD 0----------
 event_id    | 0
 event_date  | 0
 customer_id | 0
 product_id  | 0
 campaign_id | 0
 event_type  | 0
 device      | 0
 price       | 0
 quantity    | 0
 revenue     | 0
 ad_cost     | 0
 year_month  | 0
```

Figure 6. Null values per column in the fact table: none. The dimensions also have no nulls.

```
========DUPLICATE KEYS========
fact_events (event_id) duplicates: 0
dim_customers (customer_id) duplicates: 0
dim_products (product_id) duplicates: 0
dim_campaigns (campaign_id) duplicates: 0
dim_date (date_key) duplicates: 0
```

Figure 7. Duplicate keys: none in any table.

```
========REFERENTIAL INTEGRITY========
customer_id in fact_events but not in dim_customers: 0
product_id in fact_events but not in dim_products: 0
campaign_id in fact_events but not in dim_campaigns: 0
```

Figure 8. Referential integrity: no orphan customer, product or campaign keys.

```
========STATISTICS SUMMARY (fact_events)========
+-------+-----------------+-----------------+-----------------+-----------------+
|summary|            price|         quantity|          revenue|           ad_cost|
+-------+-----------------+-----------------+-----------------+-----------------+
|  count|         10000000|         10000000|         10000000|         10000000|
|   mean|255.01253447299476|         2.000005|15.303077620999932|0.2000114180000016|
| stddev| 144.3520989346148|0.8166401084107453|108.60034149745825|0.4760256577850301|
|    min|              5.0|                1|              0.0|              0.0|
|    max|            505.0|                3|           1514.97|              2.0|
+-------+-----------------+-----------------+-----------------+-----------------+
```

Figure 9. Statistics of the numeric columns of the fact table.

```
========BUSINESS RULES (fact_events)========
price not positive : 0
negative ad_cost : 0
revenue on non-purchase : 0
purchase without revenue : 0
quantity on non-purchase : 9700609
date out of range : 0
```

Figure 10. Business rules: one rule is violated, quantity on non-purchase events.

Result

Thirteen of the fourteen checks passed. The one failure is **quantity on non-purchase**: 9,700,609 events such as impressions and clicks carry a quantity of 1 to 3 although nothing was bought. This is a flaw of the data generator, and it is fixed in the processing step.

```
========DATA QUALITY REPORT========
+-----------------------------------------+-----------+------+
|check_name                               |failed_rows|status|
+-----------------------------------------+-----------+------+
|duplicate event_id in fact_events        |0          |PASS  |
|duplicate customer_id in dim_customers   |0          |PASS  |
|duplicate product_id in dim_products     |0          |PASS  |
|duplicate campaign_id in dim_campaigns   |0          |PASS  |
|duplicate date_key in dim_date           |0          |PASS  |
|orphan customer_id (fact vs dim_customers)|0         |PASS  |
|orphan product_id (fact vs dim_products) |0          |PASS  |
|orphan campaign_id (fact vs dim_campaigns)|0         |PASS  |
|price not positive                       |0          |PASS  |
|negative ad_cost                         |0          |PASS  |
|revenue on non-purchase                  |0          |PASS  |
|purchase without revenue                 |0          |PASS  |
|quantity on non-purchase                 |9700609    |FAIL  |
|date out of range                        |0          |PASS  |
+-----------------------------------------+-----------+------+
```

Figure 11. Data quality report: 13 PASS and 1 FAIL. The report is saved to HDFS.

6 Step 3: Processing with Spark

Script: `marketing_processing.py` **Input:** staging **Output:** `hdfs:///MarketingProcessed`

Cleaning the fact table

The fact table is cleaned with three rules and written as `fact_events_clean`, partitioned by month:

- keep only rows with a positive price;
- drop rows where a customer, product or campaign key is missing;
- set `quantity` to 0 when the event is not a purchase (the fix for the failed quality check).

No rows were removed (10,000,000 before and 10,000,000 after), which agrees with the quality report: the only problem was the quantity column. A duplicate-removal step was left out on purpose, because the duplicate check found no duplicate keys and the extra shuffle was too heavy for the single machine.

Summary tables

Three summary tables are built from the clean fact table and joined with the dimensions. They are small, so the later queries run in seconds instead of scanning 10 million rows.

Table	One row per	Main columns
campaign_daily_summary	campaign and day	impressions, clicks, purchases, ad_cost, revenue, ctr_pct, roas, channel, objective
customer_summary	customer	total_events, orders, total_spent, first_event, last_event, country, segment
product_summary	product	views, add_to_cart, purchases, revenue, category, brand, list_price

Two definitions are used throughout: **CTR** is clicks divided by impressions, and **ROAS** (return on ad spend) is revenue divided by ad cost. Joins with the dimensions use broadcast joins because the dimensions are small, and the number of shuffle partitions is set to 24 to suit the machine.

```
rows before: 10000000
rows after:  10000000
+--------+----------+-----------+----------+-----------+----------+-------+------+--------+-------+-------+----------+
|event_id|event_date|customer_id|product_id|campaign_id|event_type| device| price|quantity|revenue|ad_cost|year_month|
+--------+----------+-----------+----------+-----------+----------+-------+------+--------+-------+-------+----------+
|  295689|2025-10-13|     383383|      4311|        183|impression| tablet| 99.46|       0|    0.0|    0.0|   2025-10|
|     155|2025-10-11|      65200|      2137|        172|impression| tablet|187.32|       0|    0.0|    0.0|   2025-10|
|  296410|2025-10-17|     411847|      1470|        269|impression|desktop|353.97|       0|    0.0|    0.0|   2025-10|
|     243|2025-10-17|     159136|      7463|        286|impression| mobile|312.12|       0|    0.0|    0.0|   2025-10|
|  296455|2025-10-23|     248155|      3723|        131|      view| mobile|182.21|       0|    0.0|    0.0|   2025-10|
+--------+----------+-----------+----------+-----------+----------+-------+------+--------+-------+-------+----------+
only showing top 5 rows
+----------+-----------+----------+----------+-----------+------+---------+-------+-------+-------+------+
|event_date|campaign_id|   channel| objective|impressions|clicks|purchases|ad_cost|revenue|ctr_pct|  roas|
+----------+-----------+----------+----------+-----------+------+---------+-------+-------+-------+------+
|2025-10-30|        327|google_ads|   traffic|         12|     7|        4|   5.25| 2680.4|  58.33|510.55|
|2025-10-26|        439|     email|conversion|         12|     2|        1|   1.95|  31.12|  16.67| 15.96|
|2025-10-21|        394|   youtube| awareness|         22|     4|        2|   3.24| 741.06|  18.18|228.72|
|2025-10-26|        440|    tiktok|   traffic|         14|    10|        1|  12.13| 465.81|  71.43|  38.4|
|2025-10-07|        149|google_ads| awareness|         19|    10|        1|    6.7|  36.86|  52.63|   5.5|
+----------+-----------+----------+----------+-----------+------+---------+-------+-------+-------+------+
only showing top 5 rows
```

Figure 12. Processing output: row count before and after cleaning, a sample of fact_events_clean (quantity is now 0 for non-purchase events) and a sample of campaign_daily_summary.

```
+-----------+------------+------+-----------+-----------+----------+-------+-------+
|customer_id|total_events|orders|total_spent|first_event|last_event|country|segment|
+-----------+------------+------+-----------+-----------+----------+-------+-------+
|     256761|          24|     2|    1353.76| 2024-01-22|2025-12-23|     AE|    New|
|     158803|          21|     1|     581.19| 2024-01-16|2025-12-22|     SA|    New|
|     269104|          12|     0|        0.0| 2024-03-21|2025-10-11|     QA|    New|
|     453490|          17|     1|     353.49| 2024-01-16|2025-12-22|     KW|Regular|
|     160264|          22|     1|    1222.74| 2024-01-05|2025-12-30|     JO|Regular|
+-----------+------------+------+-----------+-----------+----------+-------+-------+
only showing top 5 rows
```

Figure 13. Sample of customer_summary.

```
+----------+-----+-----------+---------+--------+--------+--------+----------+
|product_id|views|add_to_cart|purchases| revenue|category|   brand|list_price|
+----------+-----+-----------+---------+--------+--------+--------+----------+
|      3918|  140|         76|       29|15854.73| Fashion|Brand_74|    373.26|
|       148|  136|         83|       30|13072.27|  Sports|Brand_80|     445.0|
|      7554|  169|         64|       30|15125.71| Fashion|Brand_34|    129.83|
|       496|  142|         62|       25|15631.51|    Home|Brand_70|    417.21|
|      6397|  154|         74|       35| 18833.7|   Books|Brand_99|    500.64|
+----------+-----+-----------+---------+--------+--------+--------+----------+
only showing top 5 rows
```

Figure 14. Sample of product_summary.

7 Step 4: Hive tables

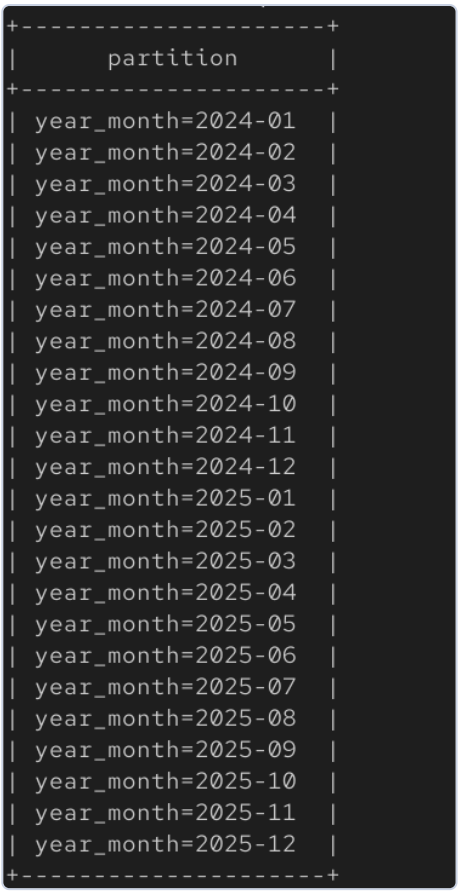
Script: `marketing_hive_tables.sql`

Hive makes the data available to SQL users without copying it. A database named `marketing` is created and eight **external** tables are defined over the Parquet files on HDFS: `fact_events_clean`, the three summary tables and the four dimensions.

- **External tables** only describe the files. Dropping a table never deletes the data on HDFS.
- **Partitions.** `fact_events_clean` is partitioned by `year_month`. After the table is created, `MSCK REPAIR TABLE` registers the 24 partitions that already exist on HDFS.
- **Metastore.** The table definitions are stored in a local Derby database (`/home/hadoop/metastore_db`), not on HDFS. Derby allows one Beeline session at a time, so any other open session must be closed first.



Figure 15. The eight external tables in the marketing database.



```
+---------------------+
|      partition      |
+---------------------+
| year_month=2024-01  |
| year_month=2024-02  |
| year_month=2024-03  |
| year_month=2024-04  |
| year_month=2024-05  |
| year_month=2024-06  |
| year_month=2024-07  |
| year_month=2024-08  |
| year_month=2024-09  |
| year_month=2024-10  |
| year_month=2024-11  |
| year_month=2024-12  |
| year_month=2025-01  |
| year_month=2025-02  |
| year_month=2025-03  |
| year_month=2025-04  |
| year_month=2025-05  |
| year_month=2025-06  |
| year_month=2025-07  |
| year_month=2025-08  |
| year_month=2025-09  |
| year_month=2025-10  |
| year_month=2025-11  |
| year_month=2025-12  |
+---------------------+
```

Figure 16. The 24 monthly partitions of fact_events_clean.

8 Step 5: Analytical queries

Script: `marketing_hive_analysis.sql`

Ten HiveQL queries answer the business questions. Most of them read the small summary tables, so they finish quickly. Each query writes its result to its own folder on HDFS as a comma-separated file without a header, using `INSERT OVERWRITE DIRECTORY`. Running the file again replaces the old results.

Folder	Question	Source
q1_funnel	How many impressions turn into clicks and purchases?	campaign_daily_summary
q2_roas_per_channel	Which channel returns the most per unit of ad spend?	campaign_daily_summary
q3_ctr_per_channel	Which channel has the highest click-through rate?	campaign_daily_summary
q4_top_campaigns	Which 10 campaigns earn the most revenue?	campaign_daily_summary
q5_revenue_per_category	Which product categories earn the most?	product_summary
q6_revenue_per_country	Which countries earn the most?	customer_summary
q7_customer_segments	How do New, Regular and VIP customers compare?	customer_summary
q8_top_customers	Who are the 10 highest-spending customers?	customer_summary
q9_weekend_vs_weekday	Do weekends sell more than weekdays?	campaign_daily_summary, dim_date
q10_monthly_revenue	How does revenue change month by month?	campaign_daily_summary, dim_date

Money columns are cast to `DECIMAL(18,2)` before saving, otherwise large values are written in scientific notation (for example 2.4E7).

```
Found 10 items
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:50 /MarketingResults/q10_monthly_revenue
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:47 /MarketingResults/q1_funnel
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:47 /MarketingResults/q2_roas_per_channel
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:48 /MarketingResults/q3_ctr_per_channel
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:48 /MarketingResults/q4_top_campaigns
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:48 /MarketingResults/q5_revenue_per_category
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:49 /MarketingResults/q6_revenue_per_country
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:49 /MarketingResults/q7_customer_segments
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:49 /MarketingResults/q8_top_customers
drwxr-xr-x   - hadoop supergroup          0 2026-10-28 05:50 /MarketingResults/q9_weekend_vs_weekday
5500785,2000859,299391,36.37,14.96
473,facebook,retargeting,353572.91,3974.53,88.96
14,facebook,conversion,353260.45,3906.05,90.44
72,tiktok,retargeting,352042.57,3895.53,90.37
348,google_ads,retargeting,347454.30,4026.56,86.29
48,tiktok,traffic,345599.57,4070.86,84.9
436,facebook,traffic,344577.78,4046.14,85.16
213,instagram,retargeting,342143.43,4214.85,81.18
376,email,awareness,342089.85,4052.19,84.42
257,email,traffic,341997.78,4018.50,85.11
84,tiktok,traffic,341565.41,3914.71,87.25
Regular,166720,51310477.37,307.76
New,166732,50896272.49,305.26
VIP,166548,50824026.35,305.16
```

Figure 17. The ten result folders on HDFS, followed by the contents of q1_funnel, q4_top_campaigns and q7_customer_segments.

9 Findings

Marketing funnel (q1)

Impressions	Clicks	Purchases	CTR	Click to purchase
5,500,785	2,000,859	299,391	36.37%	14.96%

Return on ad spend per channel (q2)

Channel	Revenue	Ad cost	ROAS
email	24,309,215.49	315,870.95	76.96
tiktok	24,870,075.88	323,455.24	76.89
instagram	25,359,670.41	331,281.03	76.55
youtube	28,754,049.33	376,937.20	76.28
facebook	24,766,723.56	324,783.95	76.26
google_ads	24,971,041.54	327,785.81	76.18

Top 10 campaigns by revenue (q4)

Campaign	Channel	Objective	Revenue	Ad cost	ROAS
473	facebook	retargeting	353,572.91	3,974.53	88.96
14	facebook	conversion	353,260.45	3,906.05	90.44
72	tiktok	retargeting	352,042.57	3,895.53	90.37
348	google_ads	retargeting	347,454.30	4,026.56	86.29
48	tiktok	traffic	345,599.57	4,070.86	84.90
436	facebook	traffic	344,577.78	4,046.14	85.16
213	instagram	retargeting	342,143.43	4,214.85	81.18
376	email	awareness	342,089.85	4,052.19	84.42
257	email	traffic	341,997.78	4,018.50	85.11
84	tiktok	traffic	341,565.41	3,914.71	87.25

Customer segments (q7)

Segment	Customers	Revenue	Average per customer
Regular	166,720	51,310,477.37	307.76
New	166,732	50,896,272.49	305.26
VIP	166,548	50,824,026.35	305.16

What the numbers say

- The funnel is healthy on paper: about 36 of every 100 impressions are clicked and about 15 of every 100 clicks end in a purchase.
- All six channels return almost the same: ROAS between 76.18 and 76.96. YouTube earns the most in total (28.75 million) because it spends the most.
- The best campaigns are retargeting and conversion campaigns on Facebook and TikTok, with ROAS between 84 and 90.
- The three customer segments are almost equal in size and in spending.

Consistency check

Total revenue is 153,030,776.21 in both the channel view (q2) and the segment view (q7), and total ad cost is 2,000,114.18, which matches the mean ad cost of the fact table in the statistics (0.2000114 per event over 10 million events). The overall ROAS is 76.51. The different views of the data agree with each other.

Note. The data is synthetic and generated with uniform random values. That is why CTR is unrealistically high and why channels and segments look almost identical. The pipeline is the result of this project, the business numbers are only a demonstration.

10 Problems and how they were solved

Problem	Cause	Solution
The virtual machine crashed during generation and processing	100 million events and heavy shuffles were too much for one machine	Reduced the fact table to 10 million events, ran Spark with 2 local cores and 2 GB memory, removed the duplicate-removal shuffle, ran queries on small summary tables
9,700,609 rows had a quantity on non-purchase events	Flaw in the data generator	Detected by the quality checks and fixed in the processing step by setting quantity to 0
Beeline reported a locked Derby database	Derby allows only one session and old Hive processes were still running	Stopped the old Hive processes, re-created the metastore and kept one Beeline session at a time
Revenue was saved in scientific notation (2.4E7)	Default formatting of double values in text files	Cast money columns to DECIMAL(18,2) before saving
Running an SQL file with `beeline -f` stopped at the first statement	Beeline reading script files does not work on this machine (Java 21)	Statements are run one at a time with `beeline -e`

11 How to run the project

All scripts are in one folder (`~/marketing`). HDFS and YARN must be running (`start-dfs.sh` and `start-yarn.sh`) and the data must already be in `/MarketingData` .

```
# 1. generate the data locally, then upload to HDFS (staging)
spark-submit generate_data.py
hdfs dfs -put ~/MarketingData/* /MarketingData/

# 2. quality checks (writes the report to /MarketingProcessed/dq_report)
spark-submit marketing_quality_checks.py

# 3. processing (writes /MarketingProcessed)
spark-submit marketing_processing.py

# 4. Hive tables:  run the statements of marketing_hive_tables.sql in Beeline
# 5. analysis:     run the statements of marketing_hive_analysis.sql in Beeline

# check the saved results
hdfs dfs -ls /MarketingResults
hdfs dfs -cat /MarketingResults/q2_roas_per_channel/*
```

Before running steps 4 and 5, close any other open Beeline session. The script `run_pipeline.sh` runs the steps from the quality checks to the analysis in order and stops at the first failure.

Files

File	Purpose
generate_data.py	Generates the star schema locally
marketing_quality_checks.py	Quality checks on the staging data and the quality report
marketing_processing.py	Cleans the fact table and builds the summary tables
marketing_hive_tables.sql	Creates the database and the eight external tables
marketing_hive_analysis.sql	Runs the ten queries and saves the results on HDFS
run_pipeline.sh	Runs the steps in order

12 Limitations and next steps

- **Synthetic data.** Distributions are uniform, so the business findings are not realistic. Real data would show differences between channels and segments.
- **Scale.** 10 million events on one machine. The same code is written to scale out to a larger cluster, but this was not tested.
- **Quality after processing.** The quantity fix was confirmed on sample rows. Running the quality checks again on `fact_events_clean` would show the failure going from 9,700,609 to 0.
- **Metastore.** Derby supports one session. A production setup would use MySQL or PostgreSQL for the Hive metastore.
- **Orchestration.** The steps are run by a shell script. A scheduler such as Airflow could run them on a schedule with retries.